Terraform with Azure

Compact Course — Day 1

Foundations & Variables

Day 1 — Agenda

Overview

- What is Terraform? How does it work?
- Advantages and disadvantages
- Downloading, installing, and finding documentation
- Project structure, `.terraform` directory, lockfiles, and `.gitignore`

Introduction

- Writing and deploying your first resource
- Formatting (`fmt`) and validating (`validate`)
- Pre-planning and provisioning to a cloud provider
- Deleting resources, state files, and idempotency
- Handling dependencies, resource addressing, `count` , and multiple variable files

Variable Types and Data Structures

- Variables, assignment, and data types (`number`, `string`, `bool`)
- Data structures: `list`, `map`, `object`, `tuple`
- Local values, string templates, conditional expressions
- `for` expressions, built-in functions, type conversions
- Using variables, input variables, outputs, and data sources
- Lifecycle meta-arguments and the `moved` block

Part 1 — Overview

What is Terraform?

- An **open-source** Infrastructure as Code (IaC) tool created by **HashiCorp**
- Uses a declarative language called **HCL** (HashiCorp Configuration Language)
- You describe the **desired state** of your infrastructure — Terraform figures out how to get there
- Supports **300+ providers**: Azure, AWS, GCP, Kubernetes, GitHub, Datadog, and more

> "Write, Plan, Apply" — infrastructure defined as code, reviewed like code, versioned like code.

What is Infrastructure as Code?

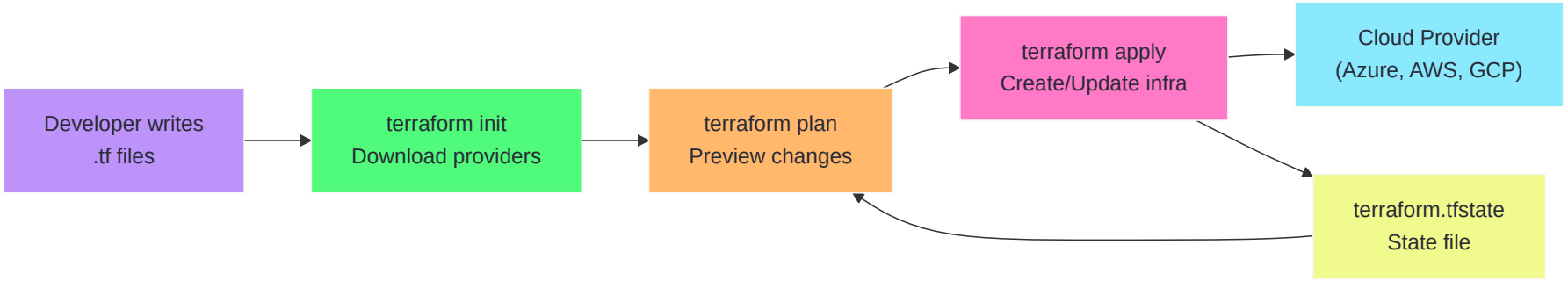
Traditional approach:

- Manually click through the Azure Portal
- Run one-off CLI scripts
- Document steps in a wiki (that's always outdated)

IaC approach:

- Infrastructure is defined in **version-controlled text files**
- Changes are **reviewed via pull requests**
- Deployments are **repeatable and auditable**
- Environments can be **reproduced exactly**

How Does Terraform Work?



How Does Terraform Work? — Step by Step

Step	Command	What happens
1	`terraform init`	Downloads provider plugins, initializes backend
2	`terraform plan`	Compares desired state (.tf) vs actual state (state file)
3	`terraform apply`	Executes the plan — creates, updates, or deletes resources
4	`terraform destroy`	Removes all resources managed by this configuration

- The **state file** (`terraform.tfstate`) is the source of truth for what Terraform manages
- The **plan** is always generated before changes are applied

Declarative vs. Imperative

	Declarative (Terraform)	Imperative (Scripts)
You describe	The end state	The steps to get there
Example	"I want 3 VMs"	"Create VM 1, create VM 2, create VM 3"
Idempotent	Yes — built in	You must handle it yourself
Drift detection	`terraform plan` shows drift	Manual comparison
Ordering	Automatic via dependency graph	You manage ordering

Advantages of Terraform

- **Multi-cloud**: single tool for Azure, AWS, GCP, and hybrid environments
- **Declarative & idempotent**: describe the end state, apply repeatedly without side effects
- **Dependency graph**: automatically determines the correct order of operations
- **Plan before apply**: preview every change before it touches your infrastructure
- **Ecosystem**: thousands of providers and modules in the Terraform Registry
- **State management**: tracks what it manages, detects drift
- **Team collaboration**: remote state, locking, and workspaces

Disadvantages & Limitations

- **State file complexity**: state must be stored securely, backed up, and locked for team use
- **Learning curve**: HCL syntax, provider-specific resources, and state management concepts
- **No rollback**: Terraform doesn't have a built-in "undo" — you revert by applying a previous config
- **Stateful**: losing or corrupting the state file can be disastrous
- **Provider lag**: new Azure features may not be available in the provider immediately
- **Secrets in state**: state files can contain sensitive values (passwords, keys) in plain text

Terraform vs. Other Tools

Feature	Terraform	ARM/Bicep	Pulumi	Ansible
Language	HCL	JSON/Bicep DSL	Python, TS, Go	YAML
Multi-cloud	✅	❌ Azure only	✅	✅
State management	✅ Built-in	❌ Azure handles it	✅ Built-in	❌ Stateless
Plan/preview	✅ `plan`	✅ `what-if`	✅ `preview`	❌
Maturity	Very high	High (Azure)	Growing	Very high
Best for	Multi-cloud IaC	Azure-native IaC	Developers (real lang)	Config mgmt

Downloading and Installing

macOS

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

Linux (Ubuntu/Debian)

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor \
  -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
  https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
  sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraform
```

Windows

```
choco install terraform
```

Verify Installation

```
$ terraform version
Terraform v1.9.x
on darwin_arm64
```

Helpful CLI commands

```
terraform -help              # List all commands
terraform plan -help         # Help for a specific command
terraform fmt                # Auto-format your .tf files
terraform validate           # Check syntax without accessing APIs
```

Finding Documentation

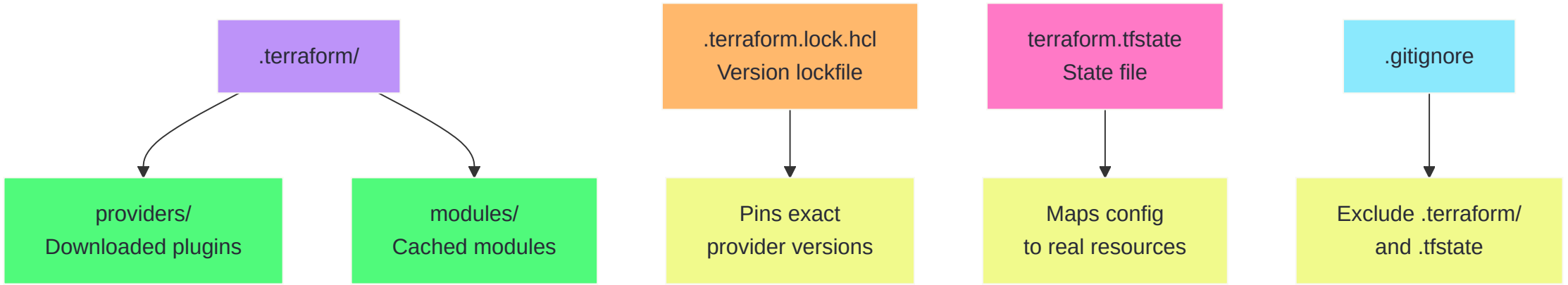
Key resources

Resource	URL
Terraform docs	`developer.hashicorp.com/terraform/docs`
Azure provider docs	`registry.terraform.io/providers/hashicorp/azurerm/latest/docs`
Terraform Registry	`registry.terraform.io`

Tips

- Every resource page shows **required** and **optional** arguments
- Look for the **Example Usage** section — copy, paste, adapt
- The **Import** section tells you how to bring existing resources under Terraform control

The `.terraform` Directory and Project Files



Understanding `.terraform/`

When you run `terraform init`, Terraform creates a `.terraform/` directory:

```
.terraform/
├── providers/
│   └── registry.terraform.io/
│       └── hashicorp/
│           └── azurerm/
│               └── 4.14.0/
│                   └── darwin_arm64/
│                       └── terraform-provider-azurerm_v4.14.0
└── modules/
    └── (cached module source code)
```

- **`providers/`** — Downloaded provider binaries (can be hundreds of MB)
- **`modules/`** — Cached module source code (from registry or Git)
- **Never commit** `.terraform/` to version control — it's regenerated by `init`

The `.terraform.lock.hcl` Lockfile

```
# This file is maintained automatically by "terraform init".
provider "registry.terraform.io/hashicorp/azurerm" {
  version     = "4.14.0"
  constraints = "~> 4.0"
  hashes = [
    "h1:aBcDeFgHiJkL...",
    "zh:0123456789ab...",
  ]
}
```

Key points

Aspect	Detail
Created by	`terraform init`
Purpose	Pins **exact** provider versions + checksums
Commit to Git?	**Yes!** Ensures the whole team uses identical versions
Update	`terraform init -upgrade` to bump within constraints

> Without the lockfile, different team members might get different provider versions.

`.gitignore` for Terraform Projects

```
# Local .terraform directories
**/.terraform/*

# .tfstate files (use remote state instead)
*.tfstate
*.tfstate.*

# Crash log files
crash.log
crash.*.log

# Exclude all .tfvars files (may contain secrets)
*.tfvars
*.tfvars.json

# Override files
override.tf
override.tf.json
*_override.tf
*_override.tf.json

# CLI configuration files
.terraformrc
terraform.rc
```

Recommended Project Structure

```
project/
├── main.tf                 ← Resource definitions
├── variables.tf            ← Input variable declarations
├── outputs.tf              ← Output value declarations
├── providers.tf            ← Provider and backend configuration
├── terraform.tfvars        ← Default variable values (gitignored)
├── dev.tfvars              ← Environment-specific values
├── prod.tfvars             ← Environment-specific values
├── .terraform.lock.hcl     ← Provider lockfile (committed)
├── .gitignore              ← Exclude .terraform/ and .tfstate
├── README.md               ← Project documentation
└── modules/                ← Local modules (if any)
    └── network/
        ├── main.tf
        ├── variables.tf
        └── outputs.tf
```

`terraform console` — Interactive Playground

```
$ terraform console
> "Hello, ${upper("terraform")}"
"Hello, TERRAFORM"

> length(["a", "b", "c"])
3

> cidrsubnet("10.0.0.0/16", 8, 1)
"10.0.1.0/24"

> merge({a=1}, {b=2}, {a=3})
{ "a" = 3, "b" = 2 }

> formatdate("YYYY-MM-DD", timestamp())
"2024-12-15"
```

- Access your variables: `var.location`
- Access data sources: `data.azurerm_subscription.current.display_name`
- Test functions before using them in code
- Exit with `exit` or Ctrl+D

> Use `terraform console` whenever you're unsure how a function or expression works.

Part 2 — Introduction

Your First Terraform Configuration

```
# main.tf

terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "example" {
  name     = "rg-terraform-demo"
  location = "West Europe"
}
```

Anatomy of a Resource Block

```
resource "azurerm_resource_group" "example" {
  name     = "rg-terraform-demo"
  location = "West Europe"
}
```

Part	Meaning
`resource`	Block type — tells Terraform to manage this
`"azurerm_resource_group"`	Resource type — maps to an Azure API
`"example"`	Local name — used to reference this resource in your code
`name`	Argument — the actual Azure resource group name
`location`	Argument — the Azure region

Pre-Planning: What to Think About Before Writing Code

Before you write HCL:

1. **What do you need?** — List the Azure resources
2. **What depends on what?** — Sketch the dependency chain
3. **What varies between environments?** — Identify variables (region, size, names)
4. **How will state be stored?** — Local for learning, remote for teams
5. **Naming conventions** — Decide on prefixes, separators, casing

> Good Terraform code starts with a clear plan on paper, not in the editor.

The Terraform Workflow

```
# 1. Initialize — download providers and set up backend
terraform init

# 2. Plan — preview what will change
terraform plan

# 3. Apply — execute the plan
terraform apply

# 4. (Optional) Destroy — tear everything down
terraform destroy
```

Each command is **safe to run** — `plan` and `init` never modify infrastructure. `apply` prompts for confirmation unless you pass `-auto-approve`.

Provisioning Your First Resource

```
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 4.0"...
- Installing hashicorp/azurerm v4.14.0...
Terraform has been successfully initialized!

$ terraform plan
  # azurerm_resource_group.example will be created
  + resource "azurerm_resource_group" "example" {
      + id       = (known after apply)
      + location = "westeurope"
      + name     = "rg-terraform-demo"
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

Applying and Verifying

```
$ terraform apply
  # azurerm_resource_group.example will be created
  + resource "azurerm_resource_group" "example" { ... }

Do you want to perform these actions?
  Enter a value: yes

azurerm_resource_group.example: Creating...
azurerm_resource_group.example: Creation complete after 2s [id=/subscriptions/.../rg-terraform-demo]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Verify in the Azure Portal or CLI:

```
az group show --name rg-terraform-demo --output table
```

`terraform fmt` — Automatic Formatting

```
$ terraform fmt
main.tf
variables.tf

$ terraform fmt -check   # CI mode — exit 1 if not formatted
$ terraform fmt -diff     # Show what would change
$ terraform fmt -recursive # Format all subdirectories too
```

What it does

- Standardizes indentation (2-space HCL convention)
- Aligns `=` signs within blocks
- Orders arguments consistently
- Does **not** change logic — purely cosmetic

> Run `terraform fmt` before every commit. Add it as a pre-commit hook or CI check.

`terraform validate` — Syntax and Logic Check

```
$ terraform validate
Success! The configuration is valid.
```

```
$ terraform validate
╷
│ Error: Missing required argument
│
│   on main.tf line 5, in resource "azurerm_resource_group" "example":
│    5: resource "azurerm_resource_group" "example" {
│
│ The argument "location" is required, but no definition was found.
╵
```

`validate` vs `plan`

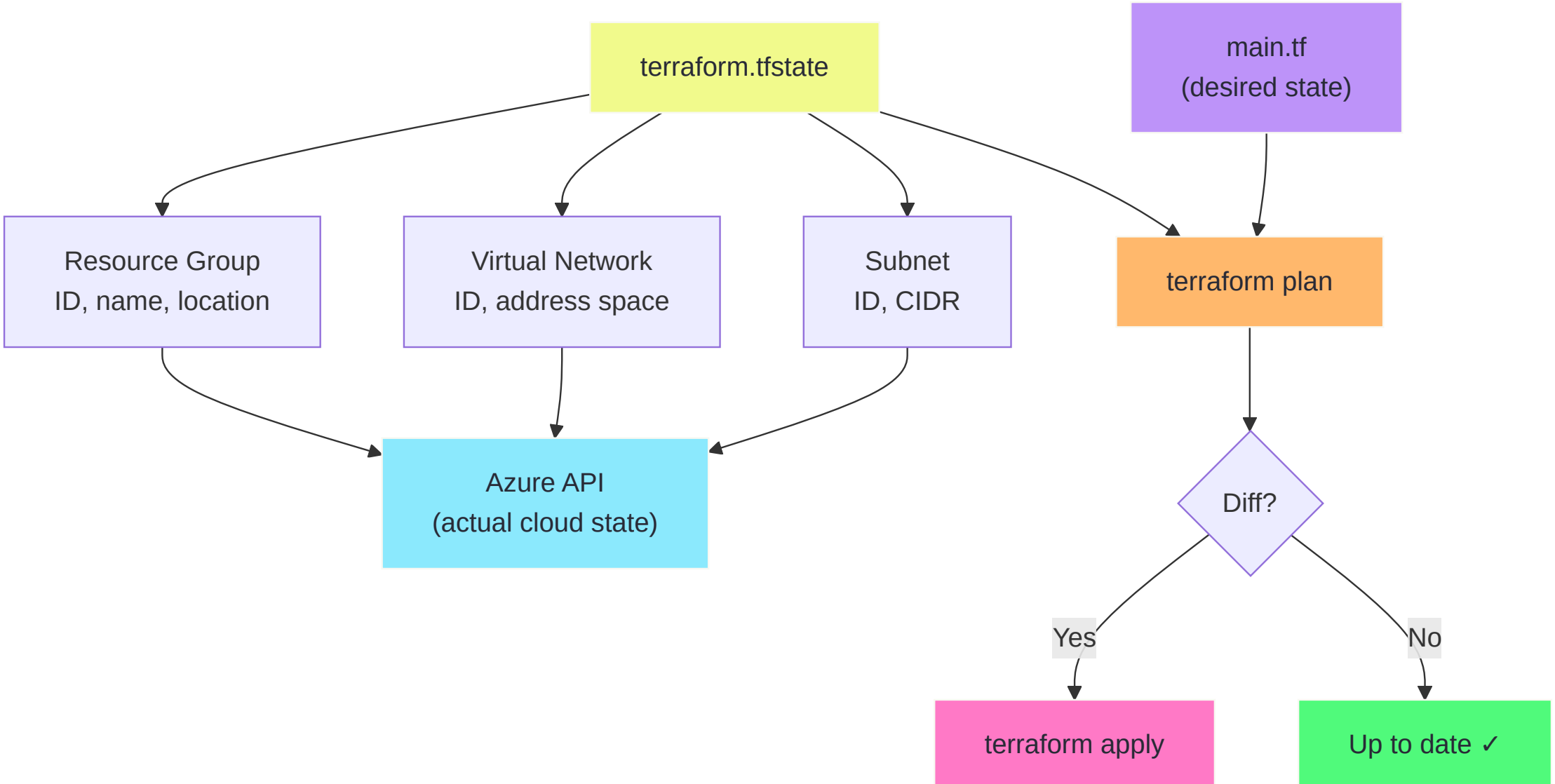
	`validate`	`plan`
Needs credentials?	No	Yes
Contacts cloud API?	No	Yes
Checks	Syntax, types, required args	Everything + actual resource state
Speed	Instant	Seconds to minutes

Deleting Resources

```
$ terraform destroy
  # azurerm_resource_group.example will be destroyed
  - resource "azurerm_resource_group" "example" {
      - id       = "/subscriptions/.../rg-terraform-demo"
      - location = "westeurope"
      - name     = "rg-terraform-demo"
    }

Plan: 0 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Enter a value: yes

Destroy complete! Resources: 1 destroyed.
```

- `destroy` removes **all resources** in the current configuration
- To remove a single resource, delete it from the `.tf` file and run `apply`

What is the State File?



State File — Key Facts

The `terraform.tfstate` file:

- Is a **JSON** file that maps your `.tf` resources to real Azure resource IDs
- Is the **single source of truth** for what Terraform manages
- Contains **sensitive data** (connection strings, passwords) — treat it as a secret!

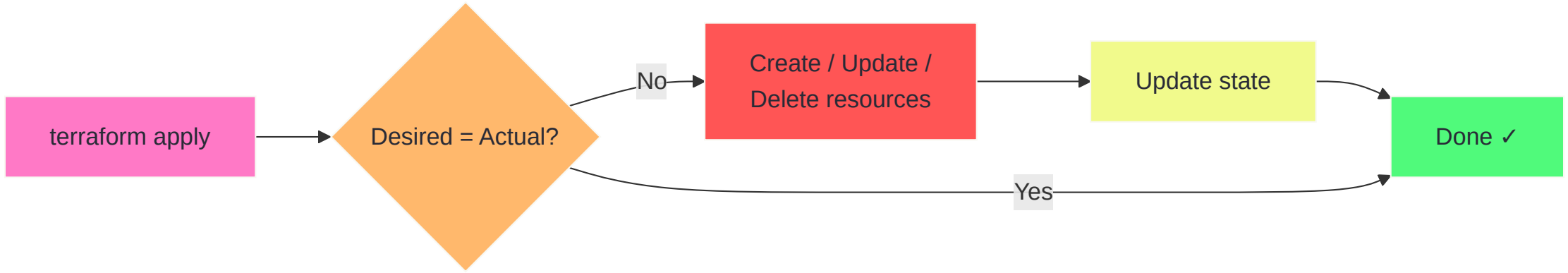
```
{
  "resources": [{
    "type": "azurerm_resource_group",
    "name": "example",
    "instances": [{
      "attributes": {
        "id": "/subscriptions/.../rg-terraform-demo",
        "location": "westeurope",
        "name": "rg-terraform-demo"
      }
    }]
  }]
}
```

State File — Rules

✅ Do	❌ Don't
Store remotely (Azure Blob) for team use	Edit the state file by hand
Enable state locking	Commit `terraform.tfstate` to Git
Use `terraform state` commands for surgery	Delete the state file without thinking
Back up regularly	Share state files via email or Slack

We'll set up **remote state** on Day 2 when we configure the Azure provider.

What is Idempotency?



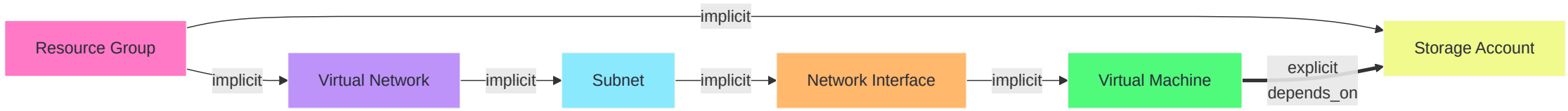
Idempotency in Practice

```
# First apply — creates the resource group
$ terraform apply
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

# Second apply — nothing changes (idempotent!)
$ terraform apply
No changes. Your infrastructure matches the configuration.
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

# Third apply — still nothing
$ terraform apply
No changes. Your infrastructure matches the configuration.
```

> **Idempotent** means you can run `apply` 100 times and the result is the same as running it once.

This is one of Terraform's biggest strengths versus imperative scripts.

Handling Dependencies



Implicit vs. Explicit Dependencies

Implicit — Terraform detects them automatically

```
resource "azurerm_virtual_network" "vnet" {
  name                = "vnet-demo"
  resource_group_name = azurerm_resource_group.example.name  # ← reference
  location            = azurerm_resource_group.example.location
  address_space       = ["10.0.0.0/16"]
}
```

The reference to `azurerm_resource_group.example.name` tells Terraform: *create the resource group first*.

Explicit — Use `depends_on` when Terraform can't detect the relationship

```
resource "azurerm_storage_account" "logs" {
  # ...
  depends_on = [azurerm_resource_group.example]
}
```

> Prefer **implicit dependencies** (references) over explicit `depends_on`.

Resource Addressing

Every resource in Terraform has a unique address:

```
<resource_type>.<local_name>
```

Address	Meaning
`azurerm_resource_group.example`	A single resource
`azurerm_subnet.subnets[0]`	Element from `count`
`azurerm_subnet.subnets["web"]`	Element from `for_each`
`module.network`	A module instance
`module.network.azurerm_virtual_network.this`	Resource inside a module
`data.azurerm_subscription.current`	A data source

Used in

- `terraform state show <address>` — inspect a resource
- `terraform state rm <address>` — remove from state
- `terraform plan -target=<address>` — plan a single resource
- References within HCL — `azurerm_resource_group.example.name`

Count — Creating Multiple Resources

```
variable "subnet_count" {
  default = 3
}

resource "azurerm_subnet" "subnets" {
  count                = var.subnet_count
  name                 = "subnet-${count.index}"
  resource_group_name = azurerm_resource_group.example.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes    = ["10.0.${count.index}.0/24"]
}
```

Expression	Result
`count.index`	0, 1, 2
`azurerm_subnet.subnets[0]`	First subnet
`azurerm_subnet.subnets[*].id`	List of all subnet IDs

Count — Things to Watch Out For

Problem: removing an item from the middle

If you have 3 subnets and delete subnet `[1]`:

- `[0]` → stays
- `[1]` → **destroyed** (was subnet-1)
- `[2]` → becomes `[1]` → **destroyed and recreated** as subnet-1!

Solution: `for_each` (covered further in the course)

```
resource "azurerm_subnet" "subnets" {
  for_each = toset(["web", "app", "db"])
  name     = "subnet-${each.key}"
  # ...
}
```

> Use `count` for identical resources. Use `for_each` when each instance is unique.

Multiple Variable Files

File structure

```
project/
├── main.tf
├── variables.tf
├── terraform.tfvars       ← auto-loaded
├── dev.tfvars             ← manually selected
└── prod.tfvars            ← manually selected
```

Usage

```
# Uses terraform.tfvars automatically
terraform apply

# Override with a specific file
terraform apply -var-file="dev.tfvars"

# Stack multiple files (last wins on conflicts)
terraform apply -var-file="common.tfvars" -var-file="prod.tfvars"
```

Multiple Variable Files — Example

```
# variables.tf
variable "environment" { type = string }
variable "location"    { type = string }
variable "vm_size"     { type = string }
```

```
# dev.tfvars
environment = "dev"
location    = "westeurope"
vm_size     = "Standard_B1s"
```

```
# prod.tfvars
environment = "prod"
location    = "westeurope"
vm_size     = "Standard_D2s_v3"
```

> Same code, different parameters — this is how you manage multiple environments.

Part 3 — Variable Types & Data Structures

Variables and Assignment

```
# variables.tf — declare the variable
variable "resource_group_name" {
  description = "Name of the resource group"
  type        = string
  default     = "rg-demo"
}

variable "location" {
  description = "Azure region"
  type        = string
  # no default → must be provided
}
```

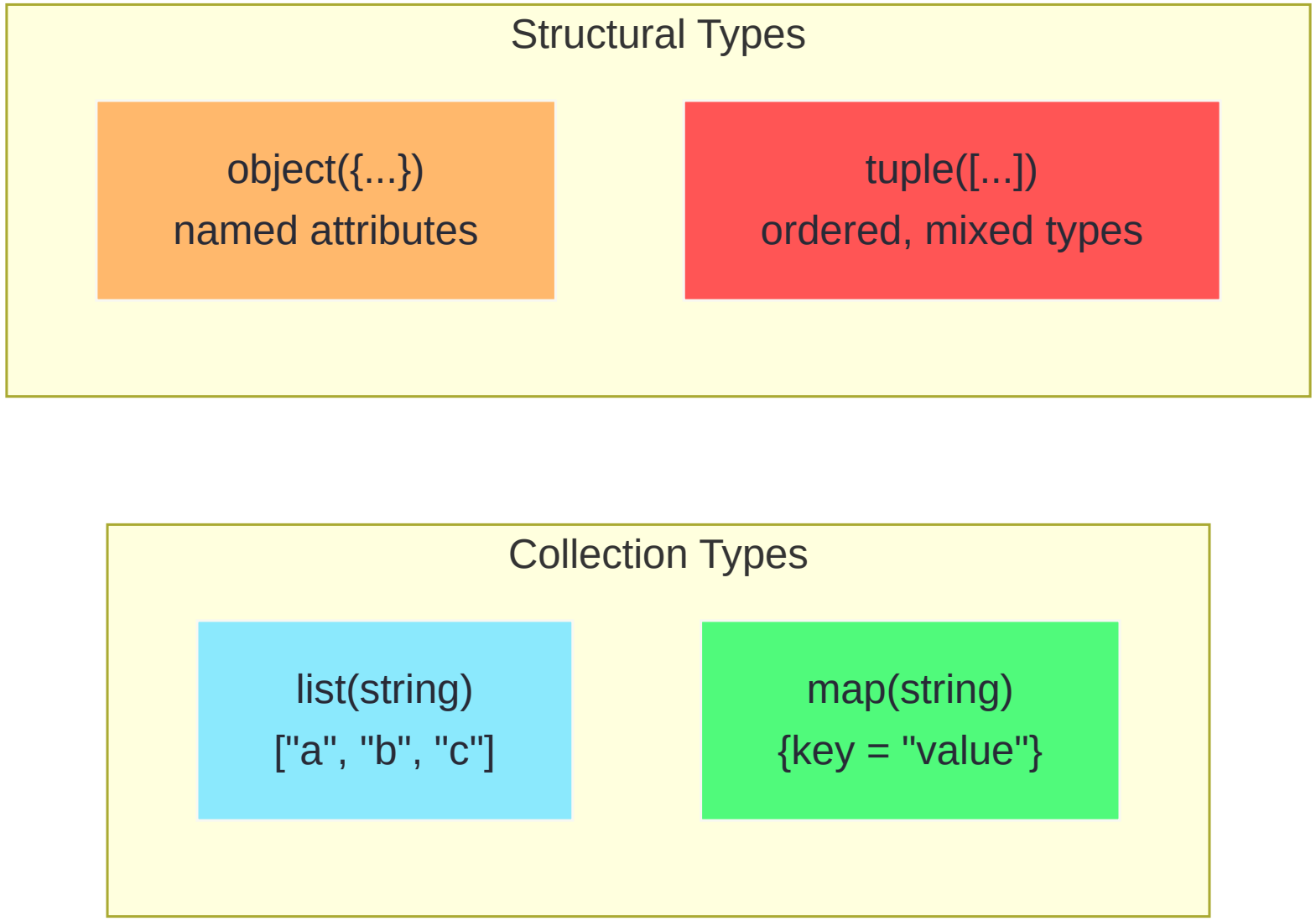
```
# main.tf – use the variable
resource "azurerm_resource_group" "example" {
  name     = var.resource_group_name
  location = var.location
}
```

Variable Blocks — Anatomy

```
variable "name" {
  description = "..."     # Optional: human-readable description
  type        = string    # Optional: type constraint
  default     = "value"   # Optional: default value
  sensitive   = true      # Optional: hide from output
  nullable    = false     # Optional: disallow null
  validation {            # Optional: custom rules
    condition     = length(var.name) > 0
    error_message = "Name must not be empty."
  }
}
```

Attribute	Required?	Purpose
`description`	No	Shows in `plan` output and docs
`type`	No (but recommended)	Enforces the variable's data type
`default`	No	Value used if none is provided
`sensitive`	No	Redacts from CLI output
`validation`	No	Custom validation logic

Terraform Variable Types



Primitive Types — `string`

```
variable "location" {
  type    = string
  default = "westeurope"
}

# Usage
resource "azurerm_resource_group" "example" {
  location = var.location                    # direct use
  name     = "rg-${var.location}-demo"       # interpolation
}
```

String functions

Function	Example	Result
`upper()`	`upper("hello")`	`"HELLO"`
`lower()`	`lower("HeLLo")`	`"hello"`
`replace()`	`replace("a-b-c", "-", "_")`	`"a_b_c"`
`substr()`	`substr("terraform", 0, 5)`	`"terra"`
`format()`	`format("rg-%s-%s", "dev", "eu")`	`"rg-dev-eu"`

Primitive Types — `number`

```
variable "instance_count" {
  type    = number
  default = 2
}

variable "disk_size_gb" {
  type    = number
  default = 128
}

# Usage
resource "azurerm_managed_disk" "data" {
  count                = var.instance_count
  disk_size_gb         = var.disk_size_gb
  name                 = "disk-data-${count.index}"
  location             = var.location
  resource_group_name  = azurerm_resource_group.example.name
  storage_account_type = "Premium_LRS"
  create_option        = "Empty"
}
```

Primitive Types — `bool`

```
variable "enable_public_ip" {
  type    = bool
  default = false
}

# Conditional expression
resource "azurerm_public_ip" "vm_pip" {
  count               = var.enable_public_ip ? 1 : 0
  name                = "pip-vm-demo"
  location            = var.location
  resource_group_name = azurerm_resource_group.example.name
  allocation_method   = "Static"
}
```

> The ternary pattern `condition ? true_value : false_value` is very common in Terraform.

Set in `.tfvars`:

```
enable_public_ip = true   # dev
enable_public_ip = false  # prod
```

Data Structures — `list`

```
variable "allowed_regions" {
  type    = list(string)
  default = ["westeurope", "northeurope", "eastus"]
}

variable "subnet_cidrs" {
  type    = list(string)
  default = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
}
```

Accessing list elements

Expression	Result
`var.allowed_regions[0]`	`"westeurope"`
`length(var.allowed_regions)`	`3`
`contains(var.allowed_regions, "eastus")`	`true`
`join(", ", var.allowed_regions)`	`"westeurope, northeurope, eastus"`
`element(var.allowed_regions, 4)`	`"northeurope"` (wraps around)

Data Structures — `map`

```
variable "tags" {
  type = map(string)
  default = {
    environment = "dev"
    project     = "terraform-training"
    owner       = "team-infra"
  }
}

# Usage — merge default tags with resource-specific ones
resource "azurerm_resource_group" "example" {
  name     = "rg-demo"
  location = "westeurope"
  tags     = merge(var.tags, {
    resource_type = "resource-group"
  })
}
```

Map functions

Function	Description
`lookup(map, key, default)`	Get value by key with fallback
`merge(map1, map2)`	Combine maps (last wins)
`keys(map)`	Get all keys as a list
`values(map)`	Get all values as a list

Data Structures — `object`

```
variable "vm_config" {
  type = object({
    name     = string
    size     = string
    os       = string
    disk_gb  = number
    public   = bool
  })
  default = {
    name    = "vm-demo"
    size    = "Standard_B2s"
    os      = "Ubuntu"
    disk_gb = 64
    public  = false
  }
}

# Usage
resource "azurerm_linux_virtual_machine" "vm" {
  name  = var.vm_config.name
  size  = var.vm_config.size
  # ...
}
```

Data Structures — `tuple`

```
variable "mixed_values" {
  type    = tuple([string, number, bool])
  default = ["westeurope", 3, true]
}

# Access by index
# var.mixed_values[0] → "westeurope"
# var.mixed_values[1] → 3
# var.mixed_values[2] → true
```

`list` vs `tuple`

	`list`	`tuple`
Element types	All **same** type	Can be **different** types
Length	Variable	**Fixed** at definition
Use case	Homogeneous collections	Ordered, typed grouping

> In practice, you'll use `list` and `object` far more often than `tuple`.

Local Values (`locals`)

```
locals {
  environment = "dev"
  location    = "westeurope"
  name_prefix = "myapp-${local.environment}"

  common_tags = {
    environment = local.environment
    managed_by  = "terraform"
    project     = "training"
  }
}

resource "azurerm_resource_group" "main" {
  name     = "${local.name_prefix}-rg"
  location = local.location
  tags     = local.common_tags
}
```

When to use `locals` vs `variables`

	`variable`	`local`
Set by	User / `.tfvars` / CLI	Computed inside code
Purpose	Input from outside	Intermediate values, DRY code
Reference	`var.name`	`local.name`

> Use `locals` to compute values from variables — keeps your resource blocks clean.

Locals — Computed Values

```
locals {
  # Build a map of subnet CIDRs from a list
  subnet_map = { for i, name in var.subnet_names :
    name => cidrsubnet(var.vnet_cidr, 8, i)
  }

  # Conditional value
  vm_size = var.environment == "prod" ? "Standard_D2s_v5" : "Standard_B1s"

  # Merge tags from multiple sources
  all_tags = merge(local.common_tags, var.extra_tags)

  # Format a name following conventions
  resource_name = lower("${var.project}-${var.environment}-${var.region}")
}
```

> `locals` are evaluated **lazily** — only computed when referenced. No performance concern for declaring many.

String Templates and Heredoc

String interpolation

```
name = "vm-${var.environment}-${var.role}"
```

Directive syntax (if/for in strings)

```
# Conditional in a string
output "message" {
  value = "Environment is ${var.environment == "prod" ? "PRODUCTION" : "non-prod"}"
}

# For loop in a string
output "subnet_list" {
  value = <<-EOT
    Subnets:
    %{ for name, cidr in var.subnets ~}
    - ${name}: ${cidr}
    %{ endfor ~}
  EOT
}
```

Heredoc syntax

```
description = <<-EOF
  This is a multi-line
  string that preserves
  newlines.
EOF
```

Conditional Expressions

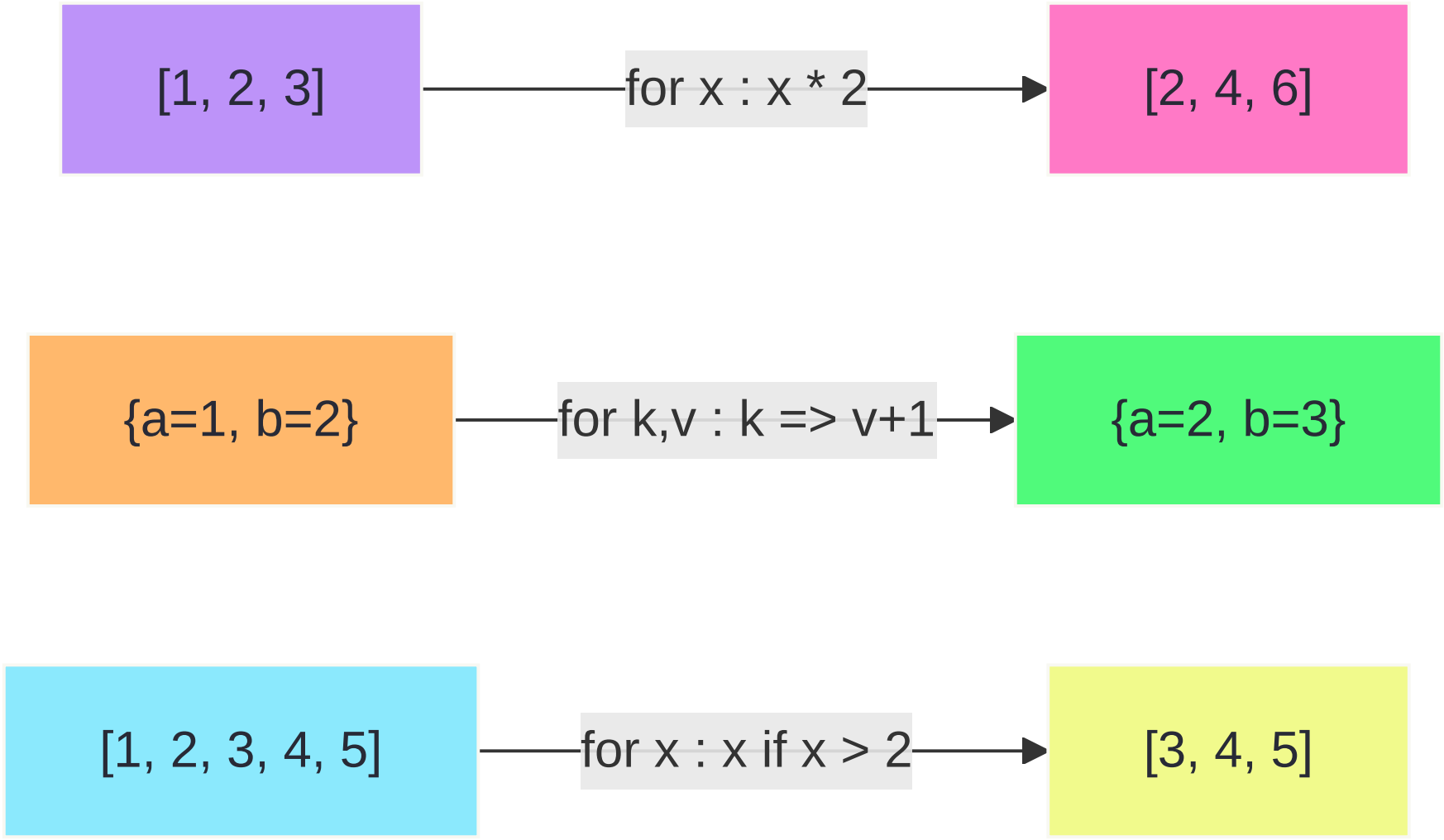
```
# Basic ternary: condition ? true_value : false_value
size = var.environment == "prod" ? "Standard_D2s_v5" : "Standard_B1s"
```

Common patterns

```
# Conditional resource creation with count
resource "azurerm_public_ip" "pip" {
  count = var.create_public_ip ? 1 : 0
  name  = "pip-${var.name}"
  # ...
}

# Conditional reference (handle the optional resource)
public_ip_id = var.create_public_ip ? azurerm_public_ip.pip[0].id : null

# Conditional value from a map
sku = lookup(
  { dev = "Standard_B1s", staging = "Standard_B2s", prod = "Standard_D2s_v5" },
  var.environment,
  "Standard_B1s"  # default fallback
)
```

> Combine `count` with ternary to create or skip entire resources based on variables.

`for` Expressions



`for` Expressions — Syntax

Transform a list → list

```
# Double every number
locals {
  doubled = [for n in [1, 2, 3] : n * 2]
  # → [2, 4, 6]
}
```

Transform a list → map

```
locals {
  subnet_ids = { for s in azurerm_subnet.subnets : s.name => s.id }
  # → { "snet-web" = "/subscriptions/...", "snet-app" = "/subscriptions/..." }
}
```

Filter

```
locals {
  prod_vms = [for vm in var.vms : vm if vm.environment == "prod"]
}
```

Transform a map → map

```
locals {
  upper_tags = { for k, v in var.tags : k => upper(v) }
  # → { environment = "DEV", project = "TRAINING" }
}
```

`for` Expressions — Practical Azure Examples

```
# Build a list of all subnet IDs from a for_each resource
locals {
  all_subnet_ids = [for k, s in azurerm_subnet.subnets : s.id]
}

# Create a map of NSG rule names to priorities
locals {
  nsg_rule_summary = {
    for rule in var.nsg_rules : rule.name => rule.priority
  }
}

# Flatten a nested structure for VM data disks
locals {
  vm_disks = flatten([
    for vm_key, vm in var.vms : [
      for disk in vm.data_disks : {
        vm_key   = vm_key
        disk_name = disk.name
        size_gb   = disk.size_gb
      }
    ]
  ])
}
```

Useful Built-in Functions

String functions

Function	Example	Result
`trimspace(" hello ")`	←	`"hello"`
`split(",", "a,b,c")`	←	`["a","b","c"]`
`join("-", ["a","b"])`	←	`"a-b"`
`regex("^rg-(.+)", "rg-demo")`	←	`["demo"]`

Collection functions

Function	Example	Result
`flatten([[1,2],[3]])`	←	`[1,2,3]`
`distinct([1,2,2,3])`	←	`[1,2,3]`
`sort(["c","a","b"])`	←	`["a","b","c"]`
`zipmap(["a","b"],[1,2])`	←	`{a=1,b=2}`

Numeric / encoding

Function	Example	Result
`min(3,1,2)`	←	`1`
`max(3,1,2)`	←	`3`
`base64encode("hello")`	←	`"aGVsbG8="`
`jsonencode({a=1})`	←	`'{"a":1}'`

Type Conversion Functions

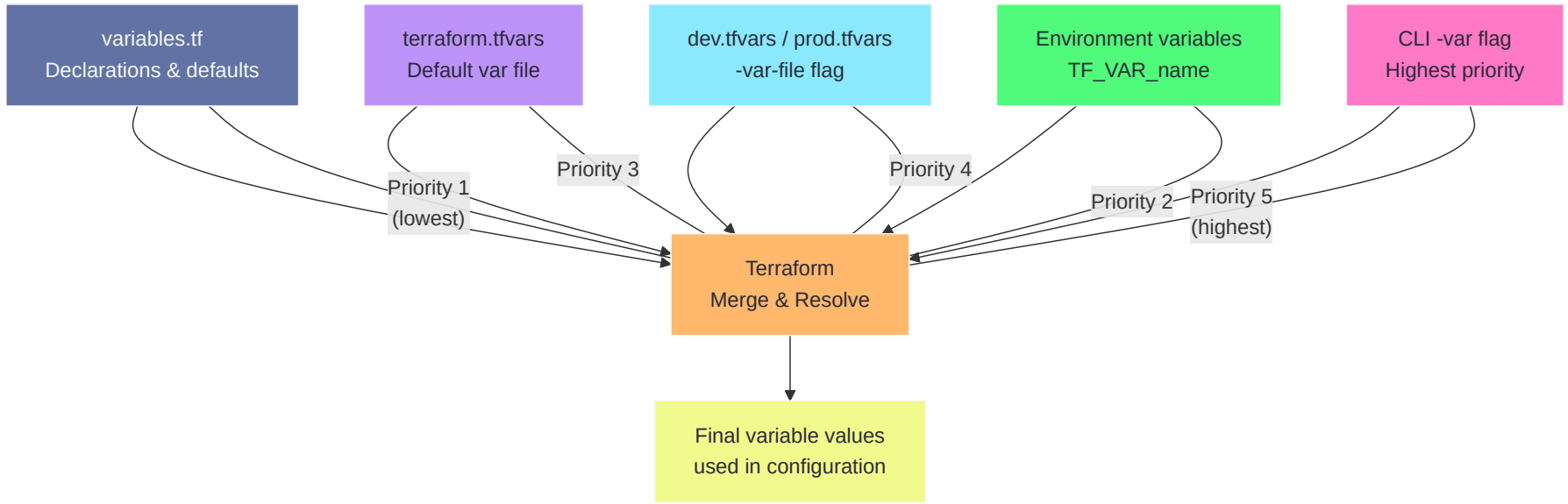
```
# Convert list to set (removes duplicates, required for for_each)
resource "azurerm_subnet" "this" {
  for_each = toset(var.subnet_names)
  name     = each.value
  # ...
}

# Convert to specific types
tostring(42)       # → "42"
tonumber("42")     # → 42
tobool("true")     # → true
tolist(toset(["a","b"]))  # set → list
tomap({a = "1"})   # ensure it's a map
```

`try()` and `can()` — Safe access

```
# try returns the first expression that doesn't error
local {
  region = try(var.config.region, "westeurope")
}

# can returns true/false
locals {
  has_region = can(var.config.region)
}
```

> `try()` is essential for working with optional object attributes.

Variable Precedence



Variable Precedence — The Rules

From **lowest** to **highest** priority:

1. `default` value in the variable block
2. Environment variables (`TF_VAR_name`)
3. `terraform.tfvars` file (auto-loaded)
4. `*.auto.tfvars` files (auto-loaded, alphabetical order)
5. `-var-file` flag on the CLI
6. `-var` flag on the CLI (highest priority)

```
# default = "westeurope" in variables.tf
# terraform.tfvars has location = "northeurope"
# but CLI flag wins:
terraform apply -var="location=eastus"
# → location = "eastus"
```

Input Variables — Best Practices

- **Always set `type`** — prevents passing the wrong data type
- **Always set `description`** — documents intent for your team
- **Use `validation` blocks** for important constraints

```
variable "environment" {
  description = "Deployment environment"
  type        = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}
```

- **Mark secrets as `sensitive`** — hides them in plan/apply output

```
variable "db_password" {
  type      = string
  sensitive = true
}
```

Outputs

```
# outputs.tf
output "resource_group_id" {
  description = "The ID of the resource group"
  value       = azurerm_resource_group.example.id
}

output "resource_group_name" {
  description = "The name of the resource group"
  value       = azurerm_resource_group.example.name
}
```

```
$ terraform apply
Apply complete!

Outputs:
resource_group_id   = "/subscriptions/.../rg-terraform-demo"
resource_group_name = "rg-terraform-demo"

$ terraform output -json
$ terraform output resource_group_id
```

> Outputs are essential for passing data **between modules** and for **CI/CD pipelines**.

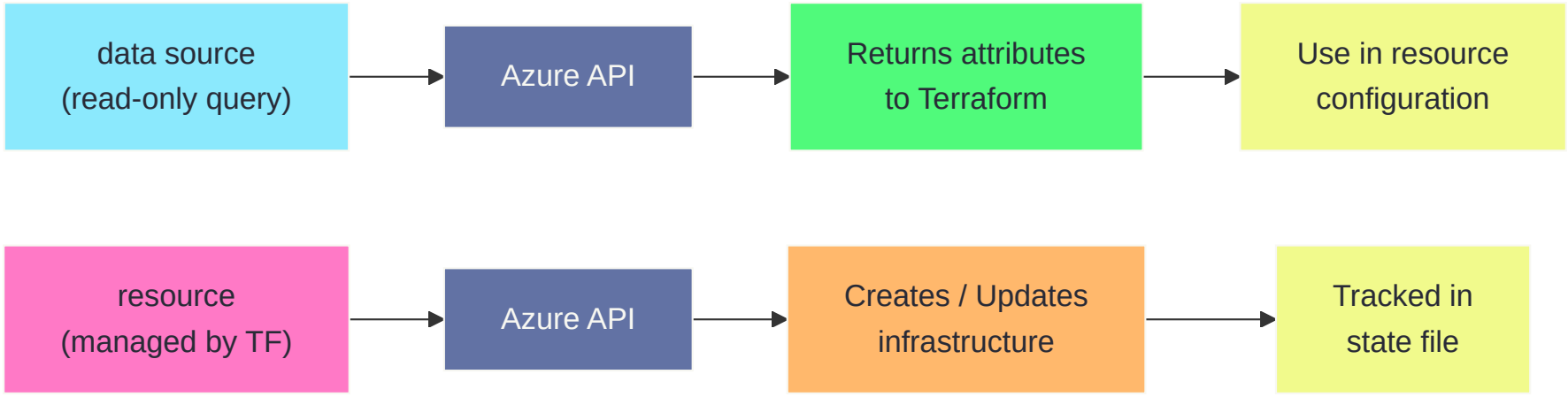
Output Blocks — Attributes

```
output "name" {
  description = "Human-readable description"
  value       = <expression>
  sensitive   = false    # hide from terminal output
  depends_on  = [...]    # rare — explicit dependency
}
```

Common patterns

```
# Single value
output "vnet_id" { value = azurerm_virtual_network.vnet.id }

# List of values from count
output "subnet_ids" { value = azurerm_subnet.subnets[*].id }

# Conditional output
output "public_ip" {
  value = var.enable_public_ip ? azurerm_public_ip.pip[0].ip_address : "none"
}
```

Data Sources



Data Sources — Reading Existing Infrastructure

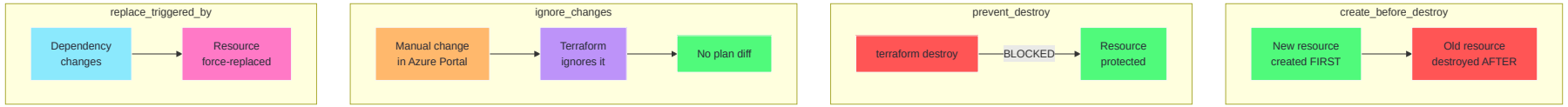
```
# Read the current Azure subscription
data "azurerm_subscription" "current" {}

# Read an existing resource group (not managed by this config)
data "azurerm_resource_group" "existing" {
  name = "rg-shared-services"
}

# Use the data in your resources
resource "azurerm_virtual_network" "vnet" {
  name                = "vnet-demo"
  resource_group_name = data.azurerm_resource_group.existing.name
  location            = data.azurerm_resource_group.existing.location
  address_space       = ["10.0.0.0/16"]
}
```

Keyword	Manages it?	In state?	Creates/deletes?
`resource`	✅ Yes	✅ Yes	✅ Yes
`data`	❌ No	Read-only	❌ No

Data Sources — Common Azure Examples

```
# Get current client config (useful for Key Vault policies, RBAC)
data "azurerm_client_config" "current" {}

# Look up a Key Vault secret
data "azurerm_key_vault_secret" "db_password" {
  name         = "db-admin-password"
  key_vault_id = data.azurerm_key_vault.kv.id
}

# Look up a VM image
data "azurerm_platform_image" "ubuntu" {
  location  = "westeurope"
  publisher = "Canonical"
  offer     = "0001-com-ubuntu-server-jammy"
  sku       = "22_04-lts"
}
```

Lifecycle Meta-Arguments



`create_before_destroy`

```
resource "azurerm_public_ip" "web" {
  name                = "pip-web-${var.environment}"
  location            = var.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method   = "Static"
  sku                 = "Standard"

  lifecycle {
    create_before_destroy = true
  }
}
```

- Default: Terraform **destroys** old, then **creates** new
- With this flag: Terraform **creates** new first, then **destroys** old
- Essential for resources that other resources depend on (public IPs, NICs, etc.)
- Reduces downtime during replacements

`prevent_destroy`

```
resource "azurerm_resource_group" "production" {
  name     = "rg-production"
  location = "westeurope"

  lifecycle {
    prevent_destroy = true
  }
}
```

```
$ terraform destroy
| Error: Instance cannot be destroyed
|
| Resource azurerm_resource_group.production has lifecycle.prevent_destroy
| set, but the plan calls for this resource to be destroyed.
```

> Use for critical resources like production databases, DNS zones, and key vaults that should never be accidentally deleted.

ignore_changes

```
resource "azurerm_linux_virtual_machine" "app" {
  name     = "vm-app"
  size     = "Standard_B2s"
  tags     = { environment = "dev" }
  # ...

  lifecycle {
    ignore_changes = [
      tags,           # Ignore manual tag changes in Portal
      custom_data,    # Ignore bootstrap script updates
    ]
  }
}
```

When to use

- **Auto-scaling** changes `instances` outside Terraform
- **Tags** managed by Azure Policy or manual intervention
- **Secrets** rotated outside Terraform
- Use `ignore_changes = all` as a last resort (ignores everything)

`replace_triggered_by`

```
resource "azurerm_linux_virtual_machine" "app" {
  name = "vm-app"
  # ...

  lifecycle {
    replace_triggered_by = [
      azurerm_network_interface.app.id,  # replace VM if NIC changes
    ]
  }
}
```

> Forces a resource to be replaced when a dependency changes, even if the resource's own attributes haven't changed.

`terraform plan -replace` (Formerly `taint`)

```
# Force recreation of a specific resource
$ terraform plan -replace="azurerm_linux_virtual_machine.app"

  # azurerm_linux_virtual_machine.app will be replaced, as requested
-/+ resource "azurerm_linux_virtual_machine" "app" { ... }

Plan: 1 to add, 0 to change, 1 to destroy.
```

When to use

- VM is in a bad state and you want to start fresh
- Resource internals are corrupt but Terraform says "no changes"
- Testing replacement behavior

> `terraform taint` is deprecated — use `-replace` flag with `plan` or `apply` instead.

The `moved` Block — Refactoring Resources

When you rename a resource or move it into a module:

```
# Before: resource "azurerm_virtual_network" "main" { ... }
# After:  resource "azurerm_virtual_network" "primary" { ... }

moved {
  from = azurerm_virtual_network.main
  to   = azurerm_virtual_network.primary
}
```

Without `moved`

- Terraform would **destroy** `main` and **create** `primary` — data loss!

With `moved`

- Terraform updates the state — **no destroy/create**, zero downtime

```
$ terraform plan
  # azurerm_virtual_network.main has moved to azurerm_virtual_network.primary
```

> Remove `moved` blocks after the migration has been applied by all environments.

Day 1 — Recap

What We Covered Today

Overview

- Terraform is a declarative, multi-cloud IaC tool using HCL
- Core workflow: `init` → `plan` → `apply` → `destroy`
- Project structure, `.terraform` directory, lockfiles, and `.gitignore`

Introduction

- Resource blocks, `fmt`, `validate`, the state file, and idempotency
- Implicit and explicit dependencies, resource addressing
- `count` for creating multiple resources
- Multiple variable files for different environments

Variables & Data Structures

- Primitives: `string`, `number`, `bool`
- Collections: `list`, `map`, `object`, `tuple`
- `locals` for computed values, string templates, conditional expressions
- `for` expressions, built-in functions, type conversions
- Variable precedence and validation
- Outputs for exposing values, data sources for reading existing infra
- Lifecycle meta-arguments and the `moved` block for safe refactoring

🛠️ Exercises

See the `exercises/day-1/` folder for four hands-on labs:

1. **Install Terraform & Create Your First Resource**
2. **Dependencies, Count, and Multiple Resources**
3. **Variables, Data Types, and Data Structures**
4. **Outputs and Data Sources**